Architecture 1001: x86-64 Assembly StructLocalVariable.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

StructLocalVariable.c

Accessing a struct local variable

```c
typedef struct mystruct{
    short a;
    int b[6];
    long long c;
} mystruct_t;

short main(){
    mystruct_t foo;
    foo.a = 0xbabe;
    foo.c = 0xba1b0ab1edb100d;
    foo.b[1] = foo.a;
    foo.b[4] = foo.b[1] + foo.c;
    return foo.b[4];
}
```

```
main:
0000000140001000  sub          rsp,38h
0000000140001004  mov          eax,0FFFFBABEh
0000000140001009  mov          word ptr [rsp],ax
000000014000100D  mov          rax,0BA1B0AB1EDB100Dh
0000000140001017  mov          qword ptr [rsp+1Ah],rax
000000014000101C  mov          eax,4
0000000140001021  imul         rax,rax,1
0000000140001025  movsx        ecx,word ptr [rsp]
0000000140001029  mov          dword ptr [rsp+rax+2],ecx
000000014000102D  mov          eax,4
0000000140001032  imul         rax,rax,1
0000000140001036  movsxd       rax,dword ptr [rsp+rax+2]
000000014000103B  add          rax,qword ptr [rsp+1Ah]
0000000140001040  mov          ecx,4
0000000140001045  imul         rcx,rcx,4
0000000140001049  mov          dword ptr [rsp+rcx+2],eax
000000014000104D  mov          eax,4
0000000140001052  imul         rax,rax,4
0000000140001056  movzx        eax,word ptr [rsp+rax+2]
000000014000105B  add          rsp,38h
000000014000105F  ret
```

StructLocalVariable.c

Accessing a struct local variable

```c
typedef struct mystruct{
    short a;
    int b[6];
    long long c;
} mystruct_t;

short main(){
    mystruct_t foo;
    foo.a = 0xbabe;
    foo.c = 0xba1b0ab1edb100d;
    foo.b[1] = foo.a;
    foo.b[4] = foo.b[1] + foo.c;
    return foo.b[4];
}
```



```
main:
0000000140001000  sub          rsp,38h
0000000140001004  mov          eax,0FFFFBABEh
0000000140001009  mov          word ptr [rsp],ax
000000014000100D  mov          rax,0BA1B0AB1EDB100Dh
0000000140001017  mov          qword ptr [rsp+1Ah],rax
000000014000101C  mov          eax,4
0000000140001021  imul         rax,rax,1
0000000140001025  movsx        ecx,word ptr [rsp]
0000000140001029  mov          dword ptr [rsp+rax+2],ecx
000000014000102D  mov          eax,4
0000000140001032  imul         rax,rax,1
0000000140001036  movsxd       rax,dword ptr [rsp+rax+2]
000000014000103B  add          rax,qword ptr [rsp+1Ah]
0000000140001040  mov          ecx,4
0000000140001045  imul         rcx,rcx,4
0000000140001049  mov          dword ptr [rsp+rcx+2],eax
000000014000104D  mov          eax,4
0000000140001052  imul         rax,rax,4
0000000140001056  movzx        eax,word ptr [rsp+rax+2]
000000014000105B  add          rsp,38h
000000014000105F  ret
```



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!

main()
frame

00000000`0014FE08	return address = 00000001`40001379
00000000`0014FE00	*16-byte-stack-alignment padding*
00000000`0014FDFC	undef (alignment padding)
00000000`0014FDF8	undef (alignment padding)
00000000`0014FDF4	c (MSBs) = 0ba1b0ab
00000000`0014FDF0	c (LSBs) = 1edb100d
00000000`0014FDEC	undef (struct alignment padding)
00000000`0014FDE8	b[5] = undef
00000000`0014FDE4	b[4] = 1edacacb
00000000`0014FDE0	b[3] = undef
00000000`0014FDDC	b[2] = undef
00000000`0014FDD8	b[1] = ffffbabe
00000000`0014FDD4	b[0] = undef
00000000`0014FDD0	padding, a = babe (2 bytes)

4-byte-granularity addresses

MS' struct alignment & padding rules here:
https://docs.microsoft.com/en-us/cpp/build/x64-software-conventions?view=msvc-160



STOP && UNCOMMENT THE PRAGMAS && DRAW A STACK DIAGRAM!



STOP && UNCOMMENT THE PRAGMAS && DRAW A STACK DIAGRAM!

main() frame

00000000`0014FE08	return address = 00000001`40001379
00000000`0014FE00	*16-byte-stack-alignment padding*
00000000`0014FDFC	undef (16 byte alignment padding)
00000000`0014FDF8	undef (16 byte alignment padding)
00000000`0014FDF4	undef (16 byte alignment padding)
00000000`0014FDF0	undef (16 byte alignment padding), c 2 MSBs = 0ba1
00000000`0014FDEC	c 4 middle bytes = b0ab1edb
00000000`0014FDE8	c 2 LSBs = 100d, b[5] 2 MSBs = undef
00000000`0014FDE4	b[5] 2 LSBs = undef, b[4] 2 MSBs = 1eda
00000000`0014FDE0	b[4] 2 LSBs = cacb, b[3] 2 MSBs = undef
00000000`0014FDDC	b[3] 2 LSBs = undef, b[2] 2 MSBs = undef
00000000`0014FDD8	b[2] 2 LSBs = undef, b[1] 2 MSBs = ffff
00000000`0014FDD4	b[1] 2 LSBs = babe, b[0] 2 MSBs = undef
00000000`0014FDD0	b[0] 2 LSBs = undef, a = babe (2 bytes)

3 2 1 0

4-byte-granularity addresses

StructLocalVariable.c Takeaways

- Fields in a struct ***must be*** stored in the same order they are defined in the high level language. And they will appear with the uppermost field at the lowest address, and all subsequent fields higher.

```
typedef struct mystruct{
    short a;
    int b[6];
    long long c;
} mystruct_t;

short main(){
    mystruct_t foo;
    foo.a = 0xbabe;
    foo.c = 0xba1b0ab1edb100d;
    foo.b[1] = foo.a;
    foo.b[4] = foo.b[1] + foo.c;
    return foo.b[4];
}
```

```
main:
0000000140001000  sub          rsp,38h
0000000140001004  mov          eax,0FFFFBABEh
0000000140001009  mov          word ptr [rsp],ax
000000014000100D  mov          rax,0BA1B0AB1EDB100Dh
0000000140001017  mov          qword ptr [rsp+1Ah],rax
000000014000101C  mov          eax,4
0000000140001021  imul         rax,rax,1
0000000140001025  movsx        ecx,word ptr [rsp]
0000000140001029  mov          dword ptr [rsp+rax+2],ecx
000000014000102D  mov          eax,4
0000000140001032  imul         rax,rax,1
0000000140001036  movsxd       rax,dword ptr [rsp+rax+2]
000000014000103B  add          rax,qword ptr [rsp+1Ah]
0000000140001040  mov          ecx,4
0000000140001045  imul         rcx,rcx,4
0000000140001049  mov          dword ptr [rsp+rcx+2],eax
000000014000104D  mov          eax,4
0000000140001052  imul         rax,rax,4
0000000140001056  movzx        eax,word ptr [rsp+rax+2]
000000014000105B  add          rsp,38h
000000014000105F  ret
```

StructLocalVariable.c Takeaways



```
typedef struct mystruct{
    short a;
    int b[6];
    long long c;
} mystruct_t;
```



```c
typedef struct mystruct{
    short a;
    int b[6];
    long long c;
} mystruct_t;
```

StructLocalVariable.c Takeaways